

Computer Organization and Assembly Language Design Document of CCP



Session: Fall 2024

Submitted By:

Muhammad Ayan Sajid	2024-CS-661
Muhammad Husnain	2024-CS-696
Fiza Shahid Khan	2024-CS-680
Shareen Asim	2024-CS-693

Submitted To:

Mr. Nadeem Iqbal

Department of Computer Science, UET Lahore, New Campus

Table of Contents

1	System Overview & Modular Architecture	3
2	Data Structure Specifications.....	4
3	Transaction Logic & Validation State Machines	5
4	Bitwise Shipment Code Generation.....	6

1 System Overview & Modular Architecture

The Warehouse Inventory and Shipment Management System is designed on a modular x86 assembly architecture. To ensure separation of concerns, the codebase is decomposed into six logical modules that interface via standardized calling conventions:

- **Module 1: Main Control (main.asm):** Manages the main application execution loop, handles user input menus, and controls program state transitions.
- **Module 2: Inventory Management (inventory.asm):** Implements database traversal, search operations, and console table-rendering functions.
- **Module 3: Transaction Engine (transaction.asm):** Processes stock adjustments (ADD, SALE, RETURN) and pricing adjustments (PRICEUPDATE) with input validation.
- **Module 4: Statistics Engine (statistics.asm):** Calculates financial indicators (valuation, revenue) and checks low-stock thresholds.
- **Module 5: Utility Engine (utility.asm):** Implements string comparisons, ascii-to-integer conversion, and the shift/rotate hashing algorithm.
- **Module 6: System Services (services.asm):** Interoperates with operating system components through Irvine32, managing file input, output generation, and date/time retrievals.

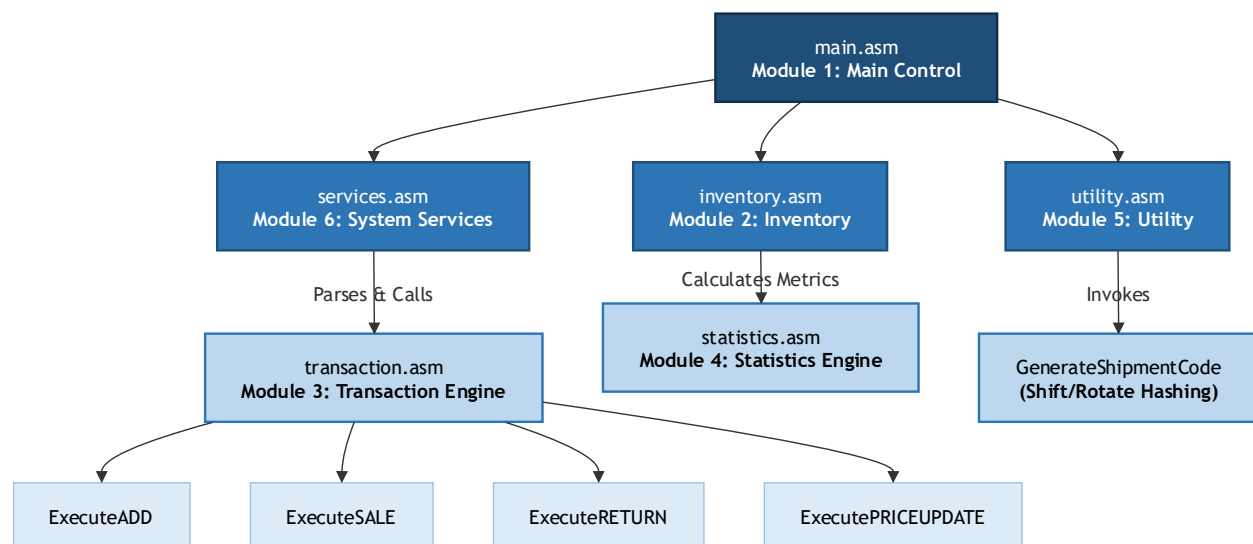
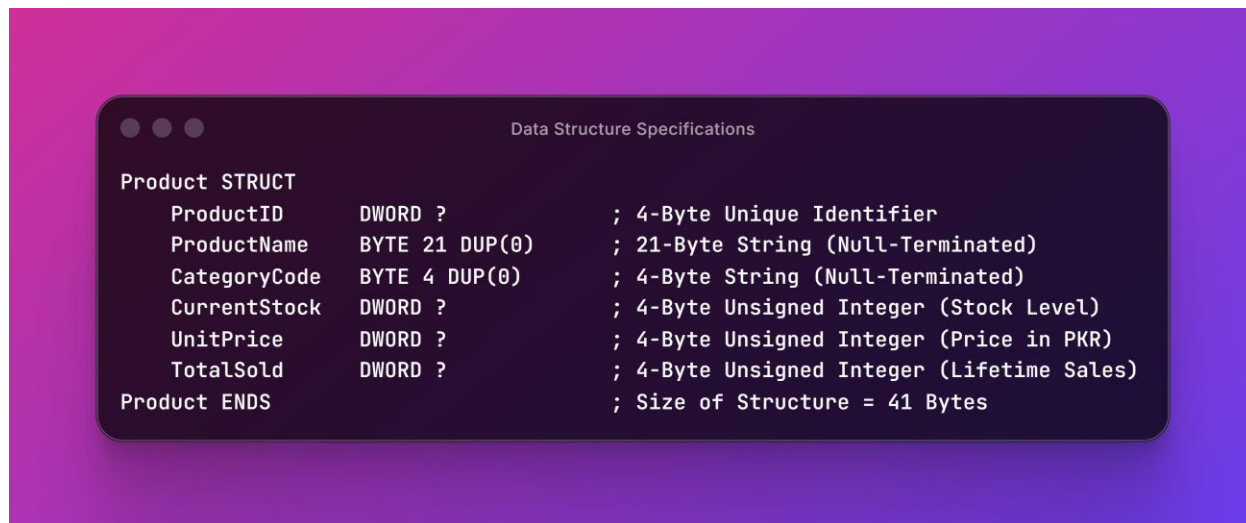


Figure 1 Procedure Hierarchy Diagram

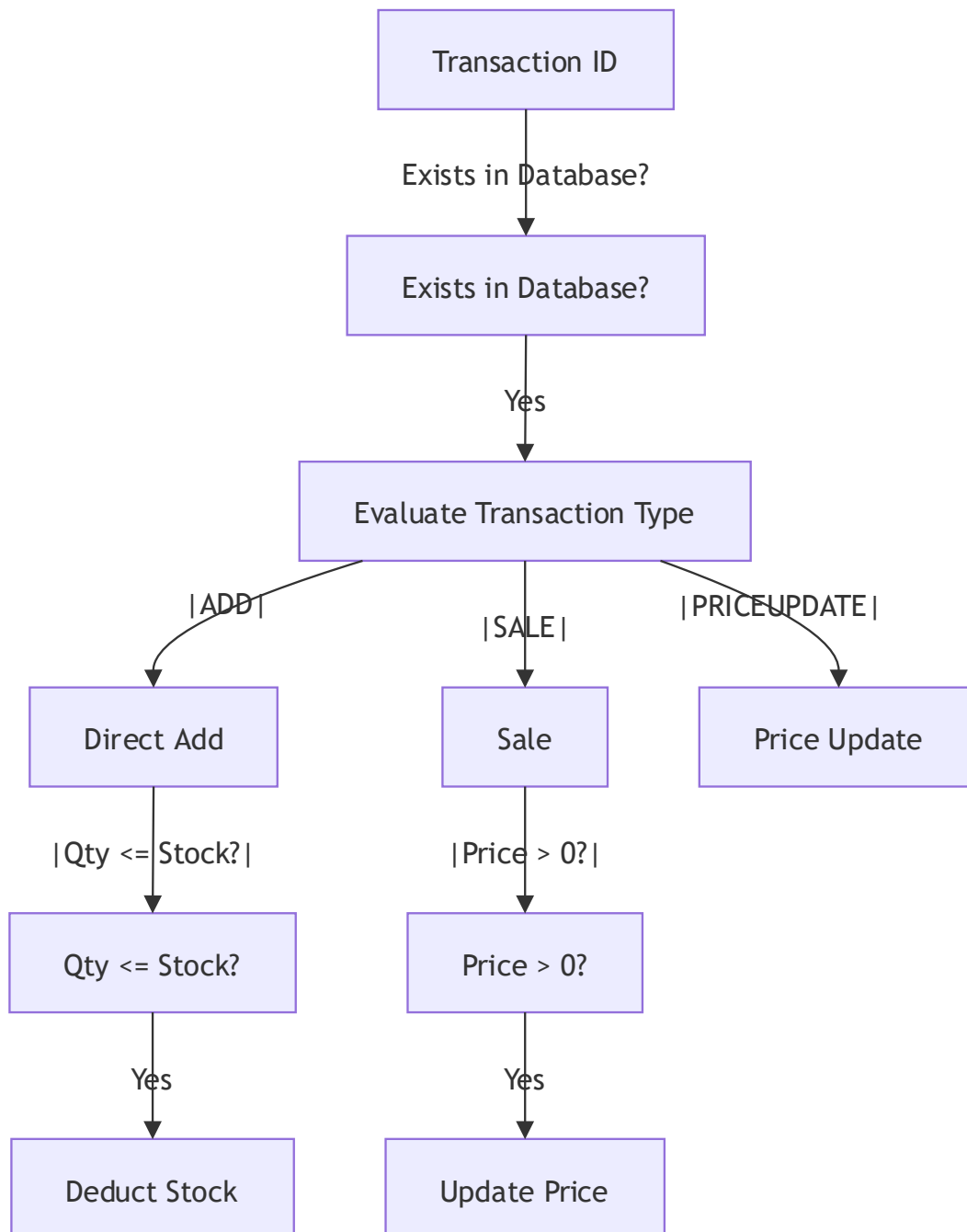
2 Data Structure Specifications

The system manages inventory records using a static array of structures (STRUCT) aligned to word boundaries.



3 Transaction Logic & Validation State Machines

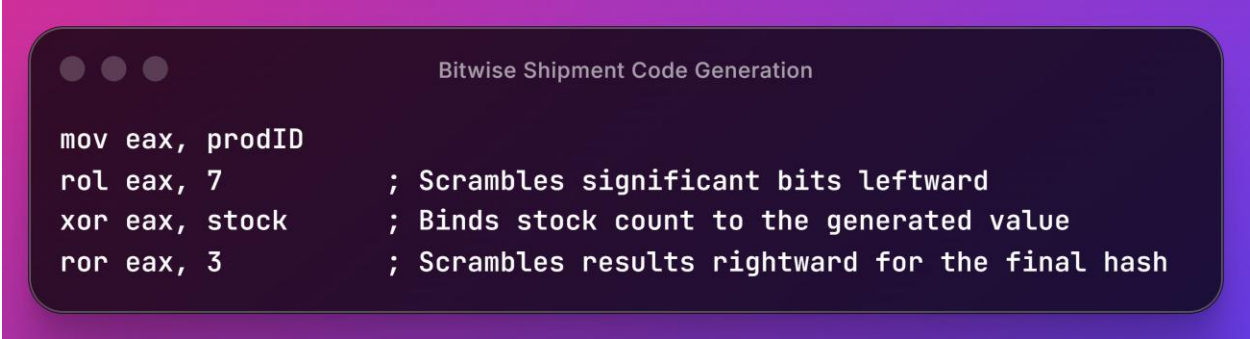
Each transaction is verified through validation rules to ensure database integrity:



1. **ADD:** Increases the CurrentStock field directly. Negative values are filtered out during standard console parsing.
2. **SALE:** Compares requested quantity against CurrentStock. If Quantity > CurrentStock, the transaction fails, preventing negative stock. Successful sales decrement CurrentStock and increment TotalSold.
3. **RETURN:** Increments CurrentStock. The system decreases TotalSold only if the total remains non-negative.
4. **PRICEUPDATE:** Verifies that the new price is greater than zero before overwriting UnitPrice.

4 Bitwise Shipment Code Generation

To generate unique tracking codes without using computationally expensive hashing libraries, the system implements bitwise shift and rotate instructions:

$$\text{Shipment Hash} = \text{RotateRight}((\text{RotateLeft}(\text{ProductID}, 7) \oplus \text{CurrentStock}), 3)$$


```
mov eax, prodID
rol eax, 7      ; Scrambles significant bits leftward
xor eax, stock  ; Binds stock count to the generated value
ror eax, 3      ; Scrambles results rightward for the final hash
```